

L01 · CORALNPU 教学仓

L01 · RV32I 单周期核

基础知识 —— 理论、术语与参考资料

对应上游仓库: [google/coralnpu](https://github.com/google/coralnpu) (RTL、文档、测试程序都来自这里)

学习方式: 读规格 → 写 RTL → `./learn check` 对拍 → 修复

文档版本: 2026-09-12 · 生成命令: `./learn pdf`

目录

- [第 1 章 这颗核要做什么](#)
 - [1.1 本课目标](#)
 - [1.2 一条指令的一生](#)
 - [1.3 和上游 CoralNPU 的关系](#)
- [第 2 章 数字逻辑与 Verilog 最小必备](#)
 - [2.1 组合逻辑与时序逻辑](#)
 - [2.2 位宽与符号扩展](#)
 - [2.3 复位与初值](#)
- [第 3 章 RV32I 指令集](#)
 - [3.1 寄存器与 ABI 命名](#)
 - [3.2 六种指令格式](#)
 - [3.3 指令语义速查](#)
- [第 4 章 内存、启动与调用约定](#)
 - [4.1 地址映射](#)
 - [4.2 小端与对齐](#)
 - [4.3 栈与函数调用](#)
 - [4.4 从 C 到你的核](#)
- [第 5 章 验收方式: 为什么是 trace 对拍](#)
- [第 6 章 自测题](#)
- [第 7 章 参考资料](#)
 - [书籍](#)
 - [规范与手册](#)
 - [教程与练习](#)
 - [本仓库内部资料](#)

第 1 章 这颗核要做什么

1.1 本课目标

L01 结束后, 你会拥有一颗自己写的 **RV32I** 单周期 **RISC-V** 核心, 它能:

- 从 ITCM (`0x0000_0000`) 取指、译码、执行、写回;
- 读写 DTCM (`0x0001_0000`) 里的全局变量和栈;

- 跑通 4 个手写汇编测试程序（算术、分支、访存、函数调用）；
- 跑通一个用 `riscv64-unknown-elf-gcc` 编译的真实 C 程序，并用 `mpause` 指令停机。

判定标准只有一条： `./learn check L01` 显示 5/5 个用例通过。

为什么从单周期开始？ 单周期核把所有事情放在一个周期里做完，没有冒险、没有流水线寄存器、没有旁路。它的价值不是性能，而是把「指令语义 → 数据通路」这条映射关系暴露得最干净。上游 CoralNPU 是 4 级流水、4 路派发、乱序退休的机器，但每一条指令的语义和你要写的一模一样。

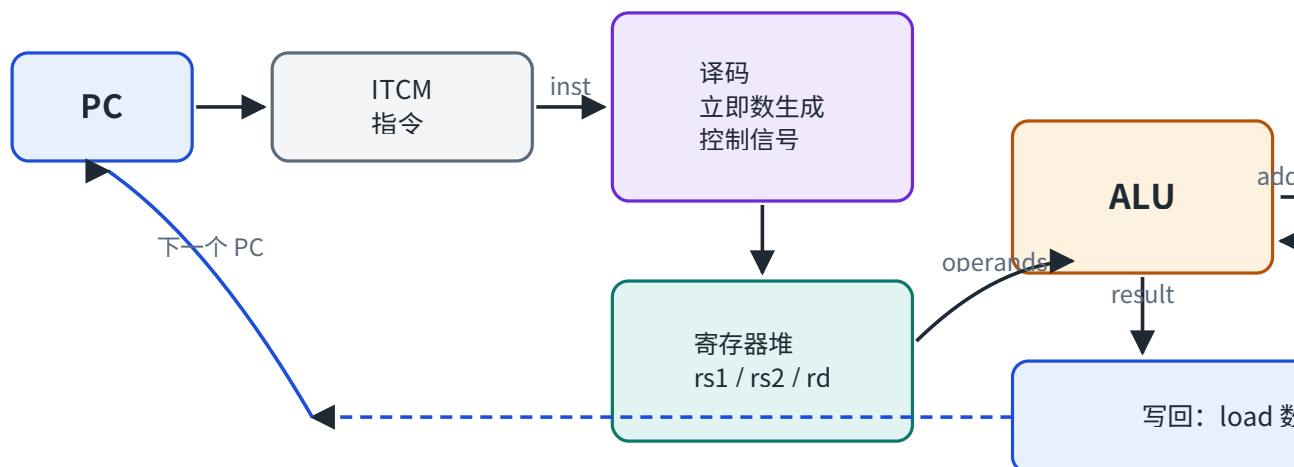
1.2 一条指令的一生

以 `addi t0, t1, 7`（把 `t1` 加 7 写入 `t0`）为例，硬件要做 5 件事：

1. **取指**：把 PC 送到指令存储器，取回 32 位机器码；
2. **译码**：从机器码里切出 `opcode` / `funct3` / `rd` / `rs1` / 立即数；
3. **读寄存器**：从寄存器堆读出 `rs1`（这里是 `t1`）；
4. **执行**：ALU 做加法（`t1 + 7`）得出结果，同时算出下一个 PC；
5. **写回**：把结果写进 `rd`（这里是 `t0`）。

流水线机器会把这 5 件事分给不同的周期、不同的硬件单元（上游 CoralNPU 就是这么做的）；单周期机器则在**同一个周期内**用一条组合逻辑长链全部做完。

单周期数据通路：一个周期走完取指、译码、执行、访存、写回



本课先实现实线部分：分支/跳转只改变 PC 的来源。（虚线回路），不影响数据流。
图 6 和 L01 骨架里的端口一一对应： `io_imem_*` / `io_dmem_*` / `o_retire_*`

单周期数据通路

图里的每个方框在 L01 骨架里都有对应的代码位置：

数据通路部件	骨架里的位置	你要做的事
PC 与下一个 PC	pc、next_pc	顺序执行是 pc+4；分支跳转改它的来源
指令存储器	io_imem_addr / io_imem_rdata	直接连 PC 即可（已给）
译码与立即数	opcode/funct3/funct7、imm_*	TODO 1、TODO 2
寄存器堆	regs[0:31]	读端口已给，写端口要接对
ALU	alu_y	TODO 3
数据存储器	io_dmem_*	TODO 5
写回	o_retire_rd / o_retire_wdata	TODO 6

1.3 和上游 CoralNPU 的关系

把上游文档摊开看，你会发现 L01 是它的「最小子集」：

项目	上游 CoralNPU	本课 L01
流水线	4 级：取指 / 译码派发 / 执行 / 写回（doc/microarch/microarch.md）	单周期，无流水
派发	4 路标量 + 2 路向量	每周期 1 条
冒险处理	记分板 + 乱序退休（doc/microarch/dispatch.md）	无（单周期天然无冒险）
访存	slot 式状态机、支持非对齐与向量访存（doc/microarch/lsu.md）	对齐访问，字节掩码
停机	mpause（Decode.scala: 1188）	相同编码 0x08000073

也就是说：你在 L01 写的译码表和 ALU，就是上游 Decode.scala 与 Alu.scala 的简化版；L03 会把流水线、冒险、乘除法、CSR 补上，L04 换成 AXI 外壳，L06/L07 再加向量与矩阵。

第 2 章 数字逻辑与 Verilog 最小必备

2.1 组合逻辑与时序逻辑

这是数字设计里最重要的分界线：

类型	特征	Verilog 写法	本课例子
组合逻辑	输出只由当前输入决定，没有记忆	<code>assign</code> 或 <code>always_comb</code>	ALU、立即数拼接、 写掩码
时序逻辑	输出在时钟边沿更新，有记忆	<code>always_ff</code> <code>@(posedge clk)</code>	PC、寄存器堆、 <code>halted</code>

三条铁律：

1. 组合逻辑里赋值用阻塞赋值 `=`，时序逻辑里用非阻塞赋值 `<=`；
2. 组合逻辑里每条被赋值的信号都要有默认值，否则综合出锁存器（latch），仿真里也会出现奇怪的保持行为；
3. 同一个信号只能由一个 `always` 块驱动。两个块都写一个变量，仿真结果依赖工具实现顺序。

第 3 条是初学者最常踩的坑：例如一个块算 `wb_data`，另一个块在 `load` 时又给 `wb_data` 赋值。正确做法是用一个 `always_comb` 做选择：`wb_data = is_load ? load_data : alu_result;`。L01 骨架已经按这个结构给你搭好了。

2.2 位宽与符号扩展

Verilog 默认按无符号处理，除非你显式声明 `signed` 或者用 `$signed()`。两个和 RISC-V 直接相关的坑：

坑一：算术右移会退化成逻辑右移。

```
y = $signed(a) >>> b; // 在无符号上下文里，很多工具会当成逻辑右移
y = (a >> b) | ({32{a[31]}} & ~(32'hFFFF_FFFF >> b)); // 稳妥：自己填符号位
```

坑二：立即数拼接。12 位立即数要符号扩展到 32 位：

```
imm_i = {{20{inst[31]}}, inst[31:20]}; // 高 20 位复制符号位
```

2.3 复位与初值

本课约定：复位有效时，PC 和全部通用寄存器清零。这样仿真结果完全可复现——黄金模型也做同样的假设，两边才能逐位对上。上游使用同步复位（见 doc/integration_guide.md 的 Reset Considerations），我们在测试平台里也按同步复位处理。

第 3 章 RV32I 指令集

3.1 寄存器与 ABI 命名

RV32I 有 32 个 32 位通用寄存器，x0 恒为 0。

寄存器	ABI 名	用途	调用约定
x0	zero	常量 0	不可写
x1	ra	返回地址	调用者保存
x2	sp	栈指针	被调用者保存
x5-x7	t0-t2	临时变量	调用者保存
x8	s0/fp	保存寄存器/帧指针	被调用者保存
x10-x11	a0-a1	参数与返回值	调用者保存
x12-x17	a2-a7	参数	调用者保存
x28-x31	t3-t6	临时变量	调用者保存

「调用者保存」的意思是：调用函数的人要自己先存好；「被调用者保存」意味着函数内部要用就得先压栈。L01 的测试 4（递归求和）会同时用到两者。

3.2 六种指令格式

RV32I 六种指令格式：位域就是硬件的接线图



图 5 立即数位域被打散是硬件布线决定的；写成拼接表达式时按图抄即可

RV32I 指令格式

位域被打散（尤其是 B 型和 J 型）不是设计缺陷，而是为了让所有指令的 `rs1` / `rs2` / `funct3` 落在相同位置，译码器可以并行读取。写代码时照抄这张图的位序即可，不要凭记忆。

3.3 指令语义速查

下面这张表就是 L01 需要实现的全部指令（`sext` 表示符号扩展）：

类别	指令	语义
U 型	lui rd, imm	rd = imm << 12
U 型	auipc rd, imm	rd = pc + (imm << 12)
跳转	jal rd, off	rd = pc+4; pc = pc + off
跳转	jalr rd, rs1, off	rd = pc+4; pc = (rs1 + off) & ~1
分支	beq/bne	相等 / 不等则跳转
分支	blt/bge	有符号比较：小于 / 大于等于
分支	bltu/bgeu	无符号比较

类别	指令	语义
载入	lb/lh/lw	读 1/2/4 字节到 rd, 符号扩展
载入	lbu/lhu	读 1/2 字节到 rd, 零扩展
存储	sb/sh/sw	把 rs2 的低 1/2/4 字节写入内存
立即数运算	addi/slti/sltiu/xori/ori/andi	rs1 op sext(imm)
立即数移位	slli/srli/srai	移位量 = imm[4:0]
寄存器运算	add/sub/sll/slt/sltu/xor/srl/sra/or/and	rs1 op rs2
其他	fence/fence.i	单周期核里当作空操作
停机	mpause / ebreak / ecall	停机 (L01 不区分原因)

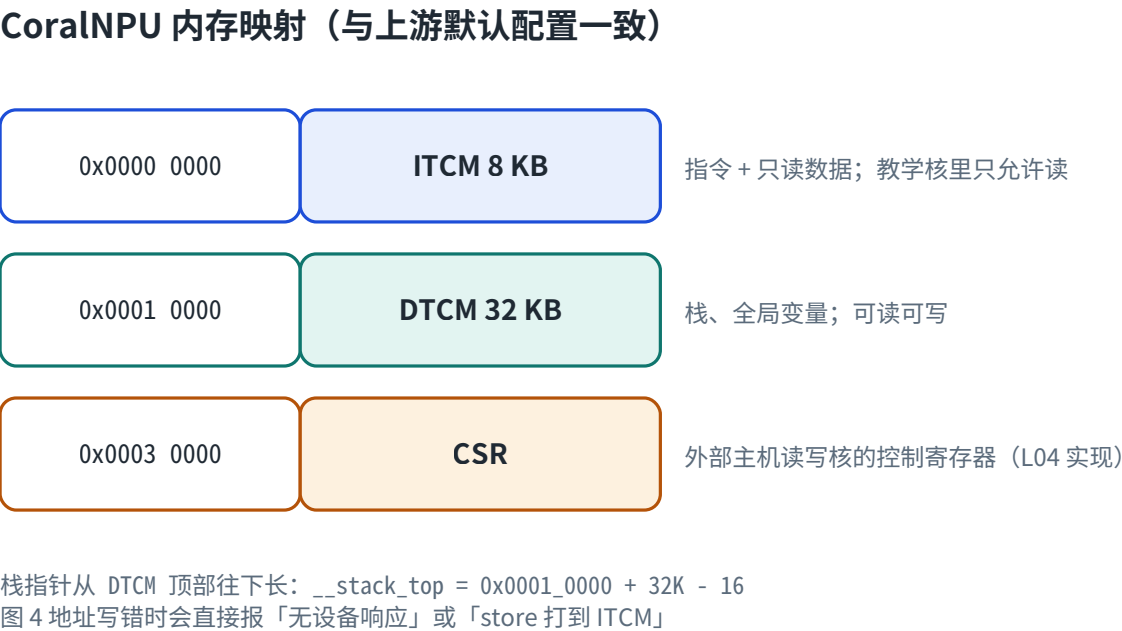
常用伪指令（汇编器展开，不占独立编码）：

伪指令	展开为
li rd, imm	lui + addi (小立即数只用 addi)
la rd, symbol	auipc + addi
mv rd, rs	addi rd, rs, 0
nop	addi x0, x0, 0
j label	jal x0, label
ret	jalr x0, 0(ra)
beqz/bnez	beq/bne rs, x0
bgt/ble	交换操作数后的 blt/bge

范围说明： 乘除法（M 扩展）、CSR（Zicsr）、浮点（F）、向量（V）都不在 L01。遇到 mul/div 时你的核应当报非法指令；它们分别在 L03、L05、L06 出现。

第 4 章 内存、启动与调用约定

4.1 地址映射



内存映射

和上游默认配置保持一致：ITCM 在 0x0000_0000（8 KB，指令与只读数据），DTCM 在 0x0001_0000（32 KB，数据与栈）。本课约定：数据端口可以读 ITCM（读常量），但不能写 ITCM。

4.2 小端与对齐

RISC-V 是小端（little-endian）：sw 把一个字写进内存时，最低字节落在最低地址。lb 0(s5) 读的是这个字的最低 8 位，lb 1(s5) 读的是第二低字节。

数据端口一次收/发一个 32 位字，所以：

- 存储：先算出地址的低 2 位，生成写掩码（sb 只写一个字节）；
- 载入：先把整字右移 8 × addr[1:0] 位，再做符号/零扩展。

非对齐访问（比如 lw 一个地址末尾是 2 的地址）在 L01 不支持，L02 才实现。

4.3 栈与函数调用

_start 会把 sp 设到 DTCM 顶部（0x0001_7FF0），栈向低地址增长。函数调用时：

```

main:
    addi    sp, sp, -4        # 申请 4 字节
    sw      ra, 0(sp)         # 保存返回地址
    call    sum_to            # ra 会被覆盖
    lw      ra, 0(sp)         # 恢复
    addi    sp, sp, 4
    ret

```

如果 **main** 里还要调用别的函数，就必须保存 **ra**——这是 L01 测试 4 故意设置的一课：忘记保存 **ra** 会让函数返回时跳回自己，表现为死循环。

4.4 从 C 到你的核

工具链流程

工具链流程

编译与链接由 `course/tools/build_program.py` 完成，它做的事都可以手工复现：

```

riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 -nostdlib -ffreestanding \
    -c course/lessons/L01_scalar/tests/link/start.S -o start.o
riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 -nostdlib -nostartfiles \
    -T course/lessons/L01_scalar/tests/link/learn_tcm.ld -o program.elf sta

```

链接脚本把 `.text` / `.rodata` 放进 ITCM、`.data` / `.bss` 放进 DTCM；启动代码 `start.S` 设置 `sp`、调用 `main`，成功返回后用 `mpause` 停机。

这比上游的 `toolchain/coralnpu_tcm.ld.tpl` 与 `toolchain/crt/coralnpu_start.S` 简单得多（去掉了 CSR 初始化、`init/fini` array、异常向量），L03 学会 CSR 与异常后可以换回上游版本。

第 5 章 验收方式：为什么是 trace 对拍

对拍：为什么它比「看波形」更快

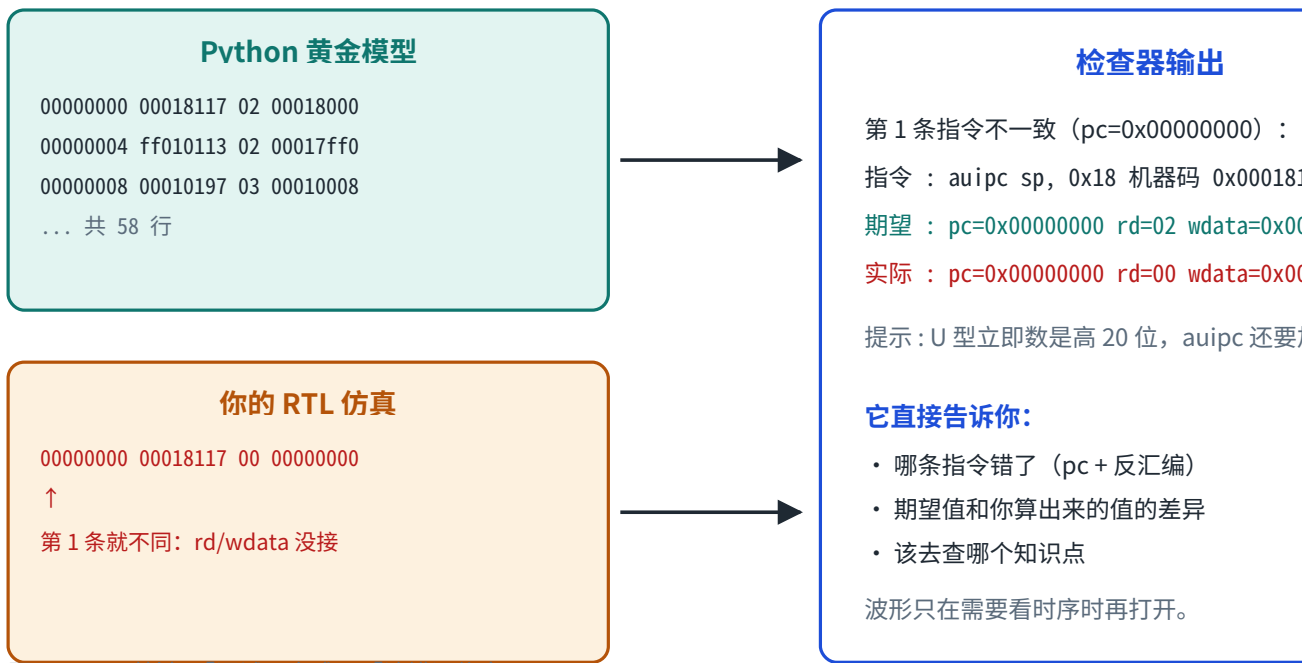


图 7 trace 对拍把「调试」变成了「读错误信息」

对拍

课程检查器会做两件事：

1. **逐指令对比 trace**：每条退休指令输出 pc 机器码 rd 写回值，两边逐行比较，第一条不一致就停下并反汇编给你看；
2. **全量对比 DTCM 8192 个字**：内存里任何一个字节写错都会暴露。

这比看波形快得多：300 条指令的程序，反馈时间不到 1 秒。波形留给「时序问题」——比如你以后做流水线时遇到的握手与冒险。

第 6 章 自测题

先自己回答，再对照括号里的提示。答不上来的，说明对应的章节要再读一遍。

1. `addi x0, x0, 5` 执行后 x0 是多少？ (3.1: x0 恒为 0)
2. `jal x0, label` 会不会写寄存器？ (不会, rd=x0)
3. B 型立即数为什么没有 bit0？ (分支目标一定是 2 字节对齐, bit0 恒为 0)
4. `srai` 和 `srli` 的机器码差在哪一位？ (bit30: funct7 的 0x20)
5. `lb 3(sp)` 在数据端口上会看到什么？ (对齐后的整字, 需要右移 24 位再取低字节)

6. sw 的写掩码为什么可能不是 4'b1111? (sw 一定是对齐的, 掩码是全 1; 但 sb/sh 要按地址低 2 位平移)
7. 为什么复位要把所有寄存器清零? (让硬件状态与黄金模型完全一致, 结果可复现)
8. 一个 always_comb 里漏写默认值会发生什么? (推断出锁存器)
9. \$signed(a) >>> b 在什么情况下会出错? (无符号上下文里退化为逻辑右移)
10. main 调用别的函数前为什么必须保存 ra? (call 会覆盖 ra)
11. 上游 CoralNPU 的停机指令是什么、编码是多少? (mpause, 0x08000073)
12. 你的核和上游在「派发能力」上的差距是什么? (上游 4 路派发、乱序退休; 本课每周期 1 条)

第 7 章 参考资料

下面这些资料的用法按顺序标注: 先读 (开课前)、边做边查、做完再看。

书籍

- **《Computer Organization and Design: RISC-V Edition》**
(Patterson & Hennessy) <https://www.elsevier.com/books/computer-organization-and-design-risc-v-edition/patterson/978-0-12-820331-6> 第 4 章讲数据通路与控制, 正是本课的单周期核; 第 4.5 节之后进入流水线, 对应 L03。先读第 4 章。
- **《Digital Design and Computer Architecture: RISC-V Edition》**
(Harris & Harris) <https://shop.elsevier.com/books/digital-design-and-computer-architecture/harris/978-0-12-820064-3> 如果你 Verilog 是零基础, 先读它的第 4 章 (组合逻辑) 与第 5 章 (时序逻辑)。先读, 补 Verilog。
- **《手把手教你设计 CPU——RISC-V 处理器篇》** (胡振波) <https://book.douban.com/subject/30236335/> 中文入门读物, 讲清楚「一个最小 RISC-V 核需要哪些模块」。边做边查。

规范与手册

- **RISC-V 非特权指令集规范** (含 RV32I 的精确语义与编码) <https://riscv.org/technical/specifications/> · 源码仓库 <https://github.com/riscv/riscv-isamanual> 遇到任何「这条指令到底怎么写」的问题, 以它为唯一权威。边做边查。
- **RISC-V Assembly Programmer's Manual** (伪指令与汇编语法) <https://github.com/riscv-non-isa/riscv-asm-manual>
- **RISC-V 指令速查** (按指令查编码) <https://msyksphinz-self.github.io/riscv-isadoc/html/rvi.html>

教程与练习

- **HDLBits** (Verilog 在线练习, 做完 Basics 与 Procedures 两节即可) https://hdlbits.01xz.net/wiki/Main_Page 先做, 做完再开始 L01 的 RTL。
- **ASIC-World Verilog 教程** (语法速查) <https://www.asic-world.com/verilog/veritut.html>
- **ChipVerify Verilog 教程** <https://www.chipverify.com/verilog/verilog-tutorial>
- **Icarus Verilog 文档** (本课使用的仿真器) <https://steveicarus.github.io/iverilog/>
- **Berkeley CS61C** (计算机体系结构公开课, 讲义与实验都在线) <https://inst.eecs.berkeley.edu/~cs61c/>

本仓库内部资料

- `doc/microarch/microarch.md`: 上游流水线与各执行单元延迟 (做完再看)
- `doc/microarch/dispatch.md`: 派发规则 (做完再看)
- `doc/microarch/lsu.md`: slot 式访存状态机, L02 的教材 (做完再看)
- `doc/integration_guide.md`: 内存映射、启动流程、CSR (L04 的教材)
- `hdl/chisel/src/coralnpu/scalar/Decode.scala`: 上游译码器, 你的译码表 vs 它的表格
- `toolchain/crt/coralnpu_start.S`: 上游启动代码, 注意它比我们的多做了什么